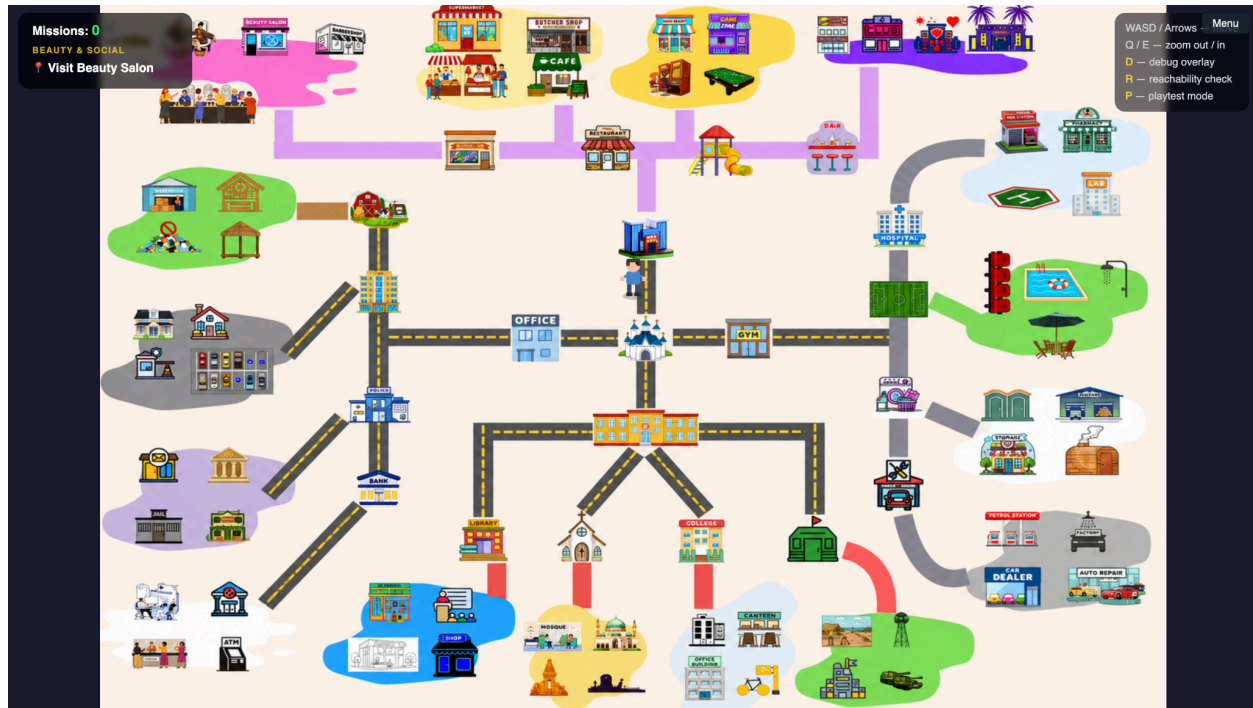


City Mission Game - Technical Summary



1. Overview

City Mission Game adalah web game 2D berbasis map PNG dan node system. Player bergerak di atas map, menerima misi menuju lokasi tertentu, lalu menyelesaikan misi ketika masuk ke radius node target.

Game ini tidak menggunakan database server, API eksternal, authentication, atau backend custom. Semua data custom level disimpan di browser menggunakan `localStorage`.

2. Tech Stack

Layer	Teknologi
Framework	Next.js 14 App Router
UI	React 18 + Tailwind CSS 3.4
Game Engine	Phaser 3.80 Canvas/WebGL

Bahasa	TypeScript 5.4 strict mode
Database	localStorage browser only
State Management	React <code>useState</code> + Phaser scene data
Deployment	Vercel
External Services	None

3. Architecture Summary

Project menggunakan kombinasi React dan Phaser.

React menangani:

- Level selection
- HUD
- Editor panel
- Custom level metadata
- Import/export JSON
- LocalStorage operations

Phaser menangani:

- Map rendering
- Player movement
- Camera
- Mission gameplay
- Node placement in editor
- Debug overlay
- Reachability check
- Playtest tracker

Phaser dijalankan hanya di client side menggunakan dynamic import dengan `ssr: false`, sehingga aman untuk Next.js SSR.

4. Gameplay Features

4.1 Player Movement

Player dapat bergerak menggunakan:

- WASD
- Arrow keys
- Virtual joystick untuk touch/mobile

Movement menggunakan hybrid speed system:

- On-road speed: 1.15x
- Off-road speed: 0.90x

4.2 Character Animation

Karakter menggunakan procedural walking sprite dengan 4-frame animation.

4.3 Mission System

Mission di-generate dari node yang valid.

Rules:

- Node dengan **purpose**: "spawn" tidak masuk mission pool
- Minimum jarak target dari player: 300px
- Menghindari district yang sama berulang
- Mission selesai saat player masuk radius target

Supported mission purposes:

- eat
- shop
- work
- rest
- visit
- deliver
- play
- social
- service

4.4 HUD

HUD menampilkan:

- Current mission
- Mission counter
- Target name
- Purpose icon/text
- Control hints

Contoh format:

 Eat at Restaurant

4.5 Camera

Camera features:

- Follow player
- Lerp follow: 0.1
- Zoom in/out dengan Q/E
- Zoom range: 0.4x - 2x
- Auto-fit saat resize

4.6 Road System

Road system digunakan untuk:

- Speed multiplier
- Walkability check
- Road proximity
- Debug analysis

Terdapat 11 segment jalan utama dengan node approach zone.

4.7 Reachability Checker

Reachability checker menggunakan Union-Find graph untuk mengecek konektivitas node terhadap road system.

Shortcut:

F1

Classification:

- Connected
- Warning
- Off-road
- No road nearby

4.8 Playtest Tracker

Playtest tracker digunakan untuk merekam 50 misi dan menghasilkan report.

Shortcut:

P

Data yang direkam:

- Mission count
- Completion time
- Target node
- District distribution
- Too fast / too slow mission
- JSON report auto-download

4.9 Debug Overlay

Debug overlay menampilkan:

- Player position
- Road status
- Speed multiplier
- Target info
- Distance to target

Shortcut:

F3 / D

4.10 Audio

Audio features:

- Background music looping
- Success SFX
- Running SFX

4.11 Mission Completion

Saat mission selesai:

- Counter bertambah
- Success popup muncul
- SFX success diputar
- Mission baru dibuat

Contoh popup:

✓ **nodeName!**

5. Node Visibility Rule

Sesuai revisi client, node tidak terlihat saat gameplay.

Gameplay Mode

Player hanya melihat:

- Map PNG original
- Player
- HUD
- Mission arrow
- Distance text

Yang disembunyikan:

- Node marker
- Node label
- Node radius
- Location assets
- Target ring default

Editor Mode

Node terlihat agar bisa diedit:

- Marker visible
- Label visible
- Radius visible
- Draggable
- Editable

Debug Mode

Debug mode dapat menampilkan informasi tambahan untuk testing.

6. Map Editor Features

Map editor terdiri dari `EditorScene` dan `EditorPanel`.

6.1 Node Management

Editor mendukung:

- Click map untuk tambah node
- Drag node untuk pindah
- Right-drag untuk pan camera
- Edit node name
- Edit node type
- Edit node purpose
- Edit radius
- Edit district
- Edit position dalam persen
- Delete node

6.2 Node Fields

Node menggunakan struktur data utama:

```
interface MapNode {  
  id: string;  
  name: string;  
  type: string;  
  purpose: string;  
  district?: string;  
  x: number;  
  y: number;  
  radius: number;  
  color?: string;  
  notes?: string;  
  icon?: string; // backward compatibility only  
  assetId?: string; // backward compatibility only  
}
```

`icon` dan `assetId` tetap ada sebagai optional backward compatibility, tetapi tidak digunakan lagi dalam UI maupun gameplay.

6.3 Mission Order

Editor mendukung mission order:

- Tambah node ke mission order
- Hapus node dari mission order
- Reorder mission
- Mission count display seperti 3/10

6.4 Level Metadata

Editor dapat mengubah:

- Level name
- Background URL
- Background upload

Background upload:

- Format: image file
- Disimpan sebagai data URL
- Maksimal 1MB

6.5 Export / Import JSON

Editor mendukung:

- Export JSON ke clipboard
- Export JSON sebagai file download
- Import JSON dari file

Import mendukung format baru dan format lama untuk backward compatibility.

6.6 Keyboard Guard

Editor memiliki guard agar shortcut game tidak aktif saat user sedang mengetik di input form.

Contoh:

- Q/E tidak trigger zoom saat mengetik
- D/R/P tidak trigger debug saat fokus di input
- WASD/Arrow tidak mengganggu text input

7. Level System

Game memiliki dua jenis level:

- Template level
- Custom level

7.1 Template Levels

Terdapat 3 template levels:

Level	Nama	Background
Level 1	Downtown	/maps/maps.png
Level 2	Suburbs	/maps/maps-lv2.png
Level 3	Industrial Zone	/maps/maps-lv3.png

Template level disimpan di:

public/levels/level1.json
 public/levels/level2.json
 public/levels/level3.json

7.2 Custom Levels

Custom levels dibuat melalui editor dan disimpan di browser menggunakan `localStorage`.

Custom level mendukung:

- Custom name
- Custom background
- Custom nodes
- Mission order
- Export/import JSON

7.3 Level Select

Level Select menampilkan:

- Template levels
- Custom levels
- Play button
- Edit button
- Delete button untuk custom level

- Create new level button

8. Data Storage

Project menggunakan `localStorage`.

Tidak ada:

- Server database
- API
- Authentication
- Cloud sync

Storage utility berada di:

`src/lib/storage.ts`

Data yang disimpan:

- Custom levels
- Recent level id
- Local template overrides jika digunakan

Catatan:

Untuk prototype, `localStorage` cukup. Jika nanti custom background besar dan banyak level, storage sebaiknya dimigrasikan ke IndexedDB.

9. Important File Structure

```
public/
├── levels/
│   ├── level1.json
│   ├── level2.json
│   └── level3.json
├── maps/
│   ├── maps.png
│   ├── maps-lv2.png
│   └── maps-lv3.png
├── music/
│   └── *.mp3
├── building/
└── road/
```

```
src/
├── app/
│   ├── page.tsx
│   └── editor/
│       └── page.tsx
├── components/
│   ├── GameCanvas.tsx
│   ├── HUD.tsx
│   ├── EditorPanel.tsx
│   ├── LevelSelect.tsx
│   └── VirtualJoystick.tsx
├── game/
│   ├── scenes/
│   │   ├── BootScene.ts
│   │   ├── MenuScene.ts
│   │   ├── GameScene.ts
│   │   └── EditorScene.ts
│   ├── systems/
│   │   ├── MapSystem.ts
│   │   ├── MissionSystem.ts
│   │   ├── CharacterSystem.ts
│   │   ├── CameraSystem.ts
│   │   ├── RoadSystem.ts
│   │   ├── EditorSystem.ts
│   │   ├── ReachabilityChecker.ts
│   │   └── PlaytestTracker.ts
│   ├── assets/
│   │   └── AssetGenerator.ts
│   ├── levels/
│   │   └── levelLoader.ts
├── lib/
└── storage.ts
```

10. Core Files

src/components/GameCanvas.tsx

Wrapper untuk Phaser instance.

Responsibilities:

- Mount Phaser game
- Pass scene data

- Prevent SSR issue
- Destroy/recreate Phaser instance when needed
- Support game/editor mode

src/components/HUD.tsx

Menampilkan:

- Mission info
- Mission counter
- Control hints
- Debug hints

src/components/EditorPanel.tsx

Sidebar editor untuk:

- Level metadata
- Node list
- Node editing
- Mission order
- Export/import
- Save/delete

src/game/scenes/GameScene.ts

Core gameplay scene.

Responsibilities:

- Load map
- Spawn player
- Start mission system
- Handle camera
- Handle keyboard shortcuts
- Handle debug overlay
- Handle mission completion
- Handle audio

src/game/scenes/EditorScene.ts

Editor scene.

Responsibilities:

- Render map background
- Show editable nodes
- Add node by click
- Drag node
- Pan camera
- Zoom camera
- Sync updates to React editor panel

src/game/systems/MapSystem.ts

Responsibilities:

- Render map background
- Render nodes in editor
- Hide nodes in gameplay
- Manage node visuals
- Support map updates

src/game/systems/MissionSystem.ts

Responsibilities:

- Generate mission
- Filter valid nodes
- Avoid spawn nodes
- Apply min distance
- Avoid repeated district
- Check mission completion

src/game/systems/CharacterSystem.ts

Responsibilities:

- Player movement
- Speed multiplier
- Character animation
- Road/off-road detection

src/game/systems/RoadSystem.ts

Responsibilities:

- Road segments
- Walkability check

- Road proximity
- Road graph data

src/game/systems/ReachabilityChecker.ts

Responsibilities:

- Analyze node-road connectivity
- Classify node status
- Render debug overlay

src/game/systems/PlaytestTracker.ts

Responsibilities:

- Track mission playtest
- Generate stats
- Export JSON report

src/game/levels/levelLoader.ts

Responsibilities:

- Fetch template levels
- Cache loaded levels
- Convert level data into game map data

src/lib/storage.ts

Responsibilities:

- Save custom level
- Load custom levels
- Delete custom level
- Save recent level id
- Safe localStorage operations

11. Deployment

Target deployment: Vercel.

Build command:

npm run build

Deployment notes:

- No custom Vercel config required
- No `vercel.json` required
- Next.js auto-detected
- Phaser is client-only through dynamic import
- Static assets are served from `public/`

12. Current Constraints

- No backend
- No authentication
- No multiplayer
- No server database
- Custom level data is browser-local
- Large image storage is limited by browser quota
- Template JSON cannot be permanently edited from browser without backend

13. Recommended Next Steps

Short Term

1. Final manual test for Level 1, Level 2, and Level 3.
2. Verify editor mode:
 - Add node
 - Drag node
 - Rename node
 - Save
 - Refresh
 - Reopen
3. Verify gameplay:
 - Nodes hidden
 - Mission HUD visible
 - Arrow works
 - Mission completion works
4. Test mobile joystick.
5. Test Vercel deployment.

Medium Term

1. Improve editor UX.

2. Add optional node color by district/type.
3. Add better mission order UI.
4. Improve map fit on mobile.
5. Add IndexedDB if custom background storage grows.
6. Add backend only if client needs account/cloud save.

14. Final Product Flow

Create Map

User creates or uploads a full PNG map.

The PNG contains all visual locations.

Create Level

User places invisible nodes on top of the PNG.

Each node stores:

- coordinate
- name
- type
- purpose
- district
- radius

Play Level

Player sees only:

- Map PNG
- Character
- HUD
- Arrow/distance guidance

The system uses invisible nodes to:

- Generate missions
- Detect target radius
- Complete missions
- Track mission progress

15. Summary

City Mission Game is a lightweight browser-based mission game built with Next.js and Phaser. The game uses map PNG backgrounds and invisible node metadata to create playable mission levels.

The current system supports:

- Template levels
- Custom levels
- Node editor
- Mission gameplay
- Debug tools
- Playtest tracker
- Export/import JSON
- Vercel deployment

The architecture is intentionally simple:

- Map image handles visuals
- Nodes handle gameplay logic
- localStorage handles custom data
- Phaser handles gameplay
- React handles UI/editor